

第六次编程实践课

软件工程与计算 I · 2026 · 南京大学

| Week 6 · 非线性数据结构 · AI代码质量与五轮框架

今日安排

知识讲解

Java 非线性数据结构：Map 与 Set

HashMap · TreeMap · LinkedHashMap · HashSet · TreeSet

AI实践 AI 代码质量与五轮开发框架

Debug 系统化 · 测试生成 · 代码审查 · 五轮对话框架

大作业 04-Eval 答案解析 · 05-Env 布置

求值器参考答案点评 · 环境与绑定任务讲解

当堂练习

练习 Lab06 · 共 2 题，独立完成

- 练习 1: HashMap 词频统计
- 练习 2: HashSet 集合运算

目标：掌握 Map 的键值查找与 Set 的去重特性，理解 `equals` / `hashCode` 在集合中的关键作用

PART 1 · Java 知识讲解

非线性数据结构：Map 与 Set

为什么需要 Map 和 Set?

数组/ArrayList 的局限

```
// 用 ArrayList 查找学生成绩
// 每次查找都需要遍历 → O(n)
for (int i = 0; i < names.size(); i++) {
    if (names.get(i).equals("Alice"))
        return scores.get(i);
}
```

- 查找效率低: $O(n)$
- 需要维护两个并行列表
- 判断"是否存在"也要遍历

Map / Set 的优势

```
// HashMap: 键值映射, O(1) 查找
HashMap<String, Integer> grades = new HashMap<>();
grades.put("Alice", 95);
grades.get("Alice"); // → 95 O(1)

// HashSet: 去重集合, O(1) 查找
HashSet<String> names = new HashSet<>();
names.add("Alice");
names.contains("Alice"); // → true O(1)
```

- 查找/判断效率 $O(1)$
- 天然去重 (Set)
- 键值关联 (Map)

HashMap — 最常用的键值映射

```
import java.util.HashMap;

HashMap<String, Integer> map = new HashMap<>();
map.put("Alice", 95);           // 插入键值对
map.put("Bob", 87);
map.put("Alice", 98);           // 相同 key 覆盖旧值

map.get("Alice");               // 98
map.get("Carol");               // null (不存在)
map.getOrDefault("Carol", 0);  // 0 (带默认值)

map.containsKey("Bob");         // true
map.containsValue(87);          // true
map.remove("Bob");              // 删除
map.size();                     // 1
```

O(1) 查找的核心：通过 `hashCode()` 计算存储位置，直接跳到目标。

HashMap — 遍历方式

三种遍历

```
// 方式 1: 遍历键
for (String key : map.keySet()) {
    System.out.println(key + "=" + map.get(key));
}

// 方式 2: 遍历值
for (Integer val : map.values()) {
    System.out.println(val);
}

// 方式 3: 遍历键值对 (推荐)
for (Map.Entry<String, Integer> e
     : map.entrySet()) {
    System.out.println(
        e.getKey() + "=" + e.getValue());
}
```

常用操作速查

方法	功能	时间
put(k, v)	插入/覆盖	O(1)
get(k)	查找	O(1)
containsKey(k)	是否含键	O(1)
remove(k)	删除	O(1)
size()	元素数	O(1)
keySet()	所有键集合	O(1)
values()	所有值集合	O(1)
entrySet()	键值对集合	O(1)

遍历用 `entrySet()` 最高效——避免 `keySet()` + `get()` 两次查找。

HashMap 实战 — 词频统计

```
static HashMap<String, Integer> wordCount(String[] words) {  
    HashMap<String, Integer> freq = new HashMap<>();  
    for (String w : words) {  
        freq.put(w, freq.getOrDefault(w, 0) + 1);  
    }  
    return freq;  
}
```

输入: ["apple", "banana", "apple", "cherry", "banana", "apple"]

输出: {apple=3, banana=2, cherry=1}

getOrDefault 模式: 首次出现默认 0, 之后每次 +1。这是 HashMap 最经典的用法。

TreeMap 与 LinkedHashMap

TreeMap — 有序 Map

```
TreeMap<String, Integer> tm = new TreeMap<>();
tm.put("banana", 2);
tm.put("apple", 3);
tm.put("cherry", 1);

// 按 key 自然排序遍历
for (String k : tm.keySet()) {
    System.out.println(k);
}
// apple → banana → cherry
```

- 基于**红黑树**, $O(\log n)$
- 键按自然顺序排序
- 额外支持 `firstKey()` `lastKey()`

LinkedHashMap — 保持插入顺序

```
LinkedHashMap<String, Integer> lhm =
    new LinkedHashMap<>();
lhm.put("banana", 2);
lhm.put("apple", 3);
lhm.put("cherry", 1);

for (String k : lhm.keySet()) {
    System.out.println(k);
}
// banana → apple → cherry (插入顺序)
```

- 内部维护双向链表
- $O(1)$ 查找 + 保持插入顺序
- 适合需要"有序 + 快速查找"的场景

Map 三兄弟对比

特性	HashMap	TreeMap	LinkedHashMap
查找时间	O(1)	O(log n)	O(1)
顺序	无序	键排序	插入顺序
null 键	允许 1 个	不允许	允许 1 个
底层结构	哈希表	红黑树	哈希表 + 链表
适用场景	通用查找	范围查询	有序缓存

默认选 HashMap——90% 的场景只需要快速查找。

HashSet — 无重复集合

```
import java.util.HashSet;

HashSet<String> set = new HashSet<>();
set.add("Alice");      // [Alice]
set.add("Bob");        // [Alice, Bob]
set.add("Alice");      // [Alice, Bob] ← 重复, 不添加

set.contains("Bob");   // true    O(1)
set.remove("Bob");     // [Alice]
set.size();            // 1
```

Set 的核心特性

特性	说明
无重复	<code>add</code> 相同元素返回 <code>false</code> ，集合不变
O(1) 查找	基于 <code>hashCode</code> ，和 <code>HashMap</code> 相同
无序	不保证遍历顺序

HashSet 底层就是 HashMap——key 存元素，value 存占位符。

TreeSet — 有序无重复集合

基本用法

```
TreeSet<Integer> ts = new TreeSet<>();
ts.add(30);
ts.add(10);
ts.add(20);
ts.add(10); // 重复, 不添加

// 自动排序遍历
for (int n : ts) {
    System.out.println(n);
}
// 10 → 20 → 30

ts.first(); // 10 (最小)
ts.last();  // 30 (最大)
```

Set 三兄弟对比

特性	HashSet	TreeSet	LinkedHashSet
时间	O(1)	O(log n)	O(1)
顺序	无序	排序	插入顺序
null	允许	不允许	允许

选择建议

- 需要去重？
- 不需排序 → HashSet (默认)
 - 需要排序 → TreeSet
 - 需要保持插入顺序 → LinkedHashSet

equals 与 hashCode — 问题演示

```
class Student { String name; int id; }
```

```
HashSet<Student> set = new HashSet<>();  
set.add(new Student("Alice", 1));  
set.contains(new Student("Alice", 1));  
// → false! 默认比较引用地址, 不是内容
```

正确做法: 重写 equals + hashCode

```
@Override  
public boolean equals(Object o) {  
    if (!(o instanceof Student)) return false;  
    Student s = (Student) o;  
    return id == s.id && Objects.equals(name, s.name);  
}  
  
@Override  
public int hashCode() { return Objects.hash(name, id); }
```

铁律: 重写 equals 必须同时重写 hashCode, 否则 HashMap/HashSet 无法正常工作。

equals 与 hashCode — 规则与大作业

三条铁律

规则	说明
equals 相等 → hashCode 必须相等	否则集合失效
hashCode 相等 → equals 不一定	冲突正常
重写 equals 必须重写 hashCode	成对出现

在大作业中的体现

LSymbol 用作 HashMap<LSymbol, LObject> 的键：

```
public class LSymbol extends LObject {
    public final String name;
    @Override
    public boolean equals(Object o) { ... }
    @Override
    public int hashCode() { ... }
}
```

没有正确的 equals / hashCode，整个解释器都无法工作。

非线性数据结构与大作业的联系

大作业场景	数据结构	用途
globals 全局符号表	HashMap<LSymbol, LObject>	O(1) 查找内建函数
Environment 环境	HashMap<LSymbol, LObject>	存储 let/let* 绑定
Token 去重	HashSet (概念上)	判断特殊字符集合
有序遍历	TreeMap (概念上)	按名称排序符号表

Project 04 → 05 的关键演变：从单一 globals HashMap 变为可嵌套的 Environment 链。

PART 2 · 大作业讲解

Project 04 Eval 答案解析 · Project 05 Env 布置

大作业进度回顾

解释器流水线

用户输入字符串

- ↓ 【01-REPL】 交互式读取 ← 已完成 ✓
- ↓ 【02-Lexer】 词法分析 ← 已完成 ✓
- ↓ 【03-AST】 语法分析 ← 已完成 ✓
- ↓ 【04-Eval】 求值 ← 答案解析 ★
- ↓ 【05-Env】 环境与绑定 ← 本次布置 ★
- ↓

输出结果

Project 04 答案 — 整体架构

Evaluator.java

— globals: HashMap<LSymbol, LObject>	← 全局符号表
— "+" → LFunction(2, lambda)	← 已提供示例
— "-" → LFunction(2, lambda)	← TODO 1
— "*" → LFunction(2, lambda)	← TODO 2
— "/" → LFunction(2, lambda)	← TODO 3
— "nil" → LList()	← 已提供
— "cons" → LFunction(2, lambda)	← TODO 4
— "car" → LFunction(1, lambda)	← TODO 5 (扩展2)
— "cdr" → LFunction(1, lambda)	← TODO 6 (扩展2)
— eval(LObject): LObject	← TODO 7 (核心)
— evalToString(LObject): String	← TODO 8 (扩展1)

设计理念：用 HashMap 存储函数而非 if-else 链——添加新函数只需一行 `globals.put(...)`。

Project 04 答案 — eval 核心代码

```
public static LObject eval(LObject obj) {
    if (obj instanceof LSymbol) {
        // 分支1: 符号 → 在 globals HashMap 中查找
        LSymbol symbol = (LSymbol) obj;
        if (globals.containsKey(symbol))
            return globals.get(symbol);
        else
            throw new RuntimeException("Unknown symbol: " + symbol.name);
    } else if (obj instanceof LList) {
        // 分支2: 列表 → 对每个元素递归求值, 再函数调用
        LList list = (LList) obj;
        if (list.content.isEmpty()) return obj;
        LList evaluated = new LList(list.content.stream()
            .map(Evaluator::eval).collect(Collectors.toList()));
        LObject func = evaluated.content.get(0);
        return ((LFunction) func).call(evaluated);
    } else {
        return obj; // 分支3: 字面量 → 返回自身
    }
}
```

Project 04 答案 — eval 三分支解析

三分支逻辑

输入类型	操作	示例
LSymbol	HashMap 查找	+ → LFunction
LList (空)	返回自身	() → ()
LList (非空)	递归求值+调用	(+ 1 2) → 3
LInteger	返回自身	42 → 42
LString	返回自身	"hi" → "hi"

核心：先递归求值每个子元素，再调用函数。

求值过程详解：(+ 1 2)

Step 1: 遍历列表 [+, 1, 2]
eval(+) → LFunction(+)
eval(1) → LInteger(1)
eval(2) → LInteger(2)

Step 2: 组成新列表
evaluated = [LFunction, 1, 2]

Step 3: 函数调用
LFunction.call(evaluated)
→ 1 + 2 = LInteger(3)

Project 04 答案 — LFunction 类

```
public class LFunction extends LObject {
    public final int arity; // 参数个数
    public final Function<LList, LObject> function; // Lambda 实现

    public LObject call(LList l) {
        if (l.content.size() != arity + 1) // +1 因为 get(0) 是函数自身
            throw new IllegalArgumentException();
        return function.apply(l);
    }
}
```

arity	表达式	列表长度	get(0)	get(1)	get(2)
1	(car lst)	2	函数	参数1	—
2	(+ 1 2)	3	函数	参数1	参数2

面试常问：为什么 `arity + 1`？因为 index 0 是函数对象本身，参数从 `get(1)` 开始。

Project 04 答案 — 算术运算与 nil

```
// 已提供的加法（作为模板）
globals.put(new LSymbol("+"), new LFunction(2,
    list -> new LInteger(
        ((LInteger) list.content.get(1)).value
        + ((LInteger) list.content.get(2)).value)));

// TODO 1-3: 照葫芦画瓢，只改运算符
globals.put(new LSymbol("-"), new LFunction(2,
    list -> new LInteger(
        ((LInteger) list.content.get(1)).value
        - ((LInteger) list.content.get(2)).value)));
// * 和 / 同理

// nil — 不是函数，是常量
globals.put(new LSymbol("nil"), new LList());
```

`nil` 直接映射为空列表 `LList()`，`eval(nil)` $\rightarrow$ `globals.get("nil")` $\rightarrow$ `LList()`。

Project 04 答案 — cons 列表构造

cons 代码

```
globals.put(new LSymbol("cons"),
  new LFunction(2, list -> {
    LObject toAdd = list.content.get(1);
    ArrayList<LObject> newList =
      new ArrayList<>();
    newList.add(toAdd);          // 新元素在前
    newList.addAll(((LList)     // 原列表在后
      list.content.get(2)).content);
    return new LList(newList);
  }));
```

关键：必须创建新 `ArrayList`，不能修改原列表（不可变）！

cons 执行过程

(cons 1 (cons 2 nil))

Step1: eval 内层 (cons 2 nil)
newList = [2] + [] = [2]
→ LList(2)

Step2: eval 外层 (cons 1 LList(2))
newList = [1] + [2] = [1, 2]
→ LList(1, 2)

参数索引

(cons	1	(cons 2 nil))
get(0)	get(1)	get(2)
函数	新元素	原列表

Project 04 答案 — 扩展1 evalToString

```
public static String evalToString(LObject obj) {
    if (obj instanceof LInteger)
        return String.valueOf(((LInteger) obj).value);
    else if (obj instanceof LString)
        return "\"" + ((LString) obj).value + "\"";
    else if (obj instanceof LSymbol)
        return ((LSymbol) obj).name;
    else if (obj instanceof LList) {
        LList list = (LList) obj;
        if (list.content.isEmpty()) return "()";
        StringBuilder sb = new StringBuilder("(");
        for (int i = 0; i < list.content.size(); i++) {
            if (i > 0) sb.append(" ");
            sb.append(evalToString(list.content.get(i))); // 递归
        }
        sb.append(")");
        return sb.toString();
    }
    return obj.toString(); // fallback
}
```

同一个模式复用三次: `tokensToString` → `astToString` → `evalToString`。

Project 04 答案 — 扩展2 car / cdr

car — 取列表第一个元素

```
globals.put(new LSymbol("car"),
    new LFunction(1, list ->
        ((LList) list.content.get(1))
            .content.get(0)));
// (car (cons 1 (cons 2 nil))) → 1
```

cdr — 去掉首元素的剩余部分

```
globals.put(new LSymbol("cdr"),
    new LFunction(1, list -> {
        List<LObject> old = ((LList)
            list.content.get(1)).content;
        return new LList(
            old.subList(1, old.size()));
    }));
// (cdr (cons 1 (cons 2 nil))) → (2)
```

名称由来

car = Contents of Address Register
cdr = Contents of Decrement Register
源自 1950s IBM 704 的硬件寄存器名称。

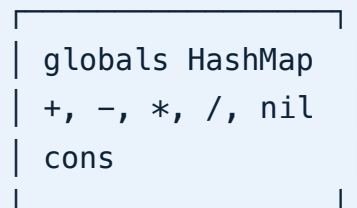
常见错误总结

错误	修正
参数取 <code>get(0)</code>	从 <code>get(1)</code> 起
cons 修改原列表	创建新 ArrayList
eval 无限递归	字面量直接返回
nil 当函数调用	nil 是 <code>LList()</code>

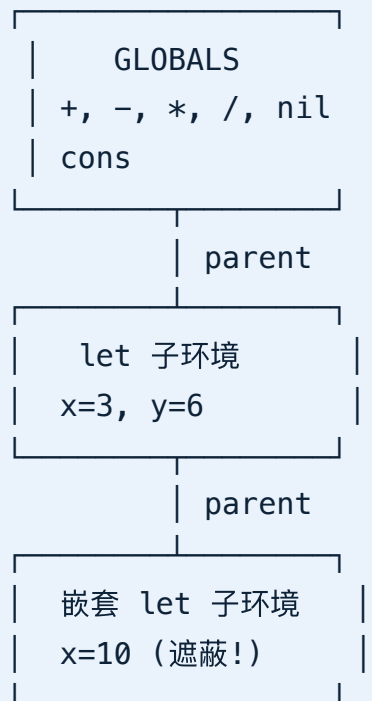
Project 05 — Env 环境与绑定

从全局 HashMap 到可嵌套的 Environment 链。

Project 04:



Project 05:



Project 05 — Environment 代码

```
public class Environment {
    private final Environment parent;
    private final HashMap<LSymbol, LObject> symbols = new HashMap<>();

    public Environment() { this.parent = null; }
    public Environment(Environment parent) { this.parent = parent; }

    public void define(LSymbol s, LObject v) { symbols.put(s, v); }

    public LObject lookup(LSymbol s) {
        if (symbols.containsKey(s)) return symbols.get(s);
        if (parent != null) return parent.lookup(s); // 递归查父
        throw new IllegalArgumentException("Invalid symbol: " + s);
    }
}
```

方法	功能
define	在当前环境中添加绑定
lookup	沿 parent 链递归向上查找

Environment 内部就是 **HashMap** + **递归查找** = 作用域链。

Project 05 — 查找与遮蔽

查找过程示例

```
(let ((x 3)) (let ((y 5)) (+ x y)))
```

lookup(y):
子{y=5} → 找到! 返回 5

lookup(x):
子{y=5} → 没有
→ 父{x=3} → 找到! 返回 3

lookup(+):
子 → 没有 → 父 → 没有
→ GLOBALS → 找到!

遮蔽 (Shadowing)

```
(let ((x 1))  
  (let ((x 2))  
    x))      → 2
```

子环境 `x=2` 遮蔽父环境 `x=1`。

核心变化 vs Project 04

	Project 04	Project 05
查找	<code>globals.get()</code>	<code>env.lookup()</code>
签名	<code>eval(obj)</code>	<code>eval(obj, env)</code>
特殊形式	无	<code>let/let*/if/begin</code>

Project 05 — let 与 let* 的区别

```
(let ((x 2) (y 3)) (let ((x 7) (z (+ x y))) (* z x))) → 35
(let ((x 2) (y 3)) (let* ((x 7) (z (+ x y))) (* z x))) → 70
```

步骤	let （旧环境求值）	let* （新环境求值）
绑定 x=7	newEnv: {x=7}	newEnv: {x=7}
求值 (+ x y)	在 env 中: x=2, y=3 → z=5	在 newEnv 中: x=7, y=3 → z=10
求值 (* z x)	z=5, x=7 → 35	z=10, x=7 → 70

inplace 参数统一两个方法——let 用旧 env 求值绑定，let* 用新 newEnv。

Project 05 — 扩展1: if 条件求值

语法: (if cond then else)

值	真假	示例
nil / ()	假	(if nil 10 20) → 20
整数 (含 0)	真	(if 0 10 20) → 10
字符串	真	(if "hi" 10 20) → 10
非空列表	真	(if (cons 1 nil) 10 20) → 10

只有空列表是"假"! 0 也是"真"。

提示代码

```
private static LObject handleIf(
    LList list, Environment env) {
    LObject cond = eval(
        list.content.get(1), env);
    boolean isFalse =
        (cond instanceof LList)
        && ((LList) cond).content.isEmpty();
    if (!isFalse)
        return eval(list.content.get(2), env);
    else
        return eval(list.content.get(3), env);
}
```

注意: if 是特殊形式——只求值一个分支, 不像普通函数先求值所有参数。

Project 05 — 扩展2: begin 顺序求值

语法

```
(begin expr1 expr2 ... exprN)
```

依次求值每个表达式, 返回**最后一个**的结果。

```
(begin 1 2 3)           → 3  
(begin (+ 1 2) (* 3 4)) → 12
```

提示代码

```
private static LObject handleBegin(LList list, Environment env) {  
    LObject result = new LList();  
    for (int i = 1; i < list.content.size(); i++) {  
        result = eval(list.content.get(i), env);  
    }  
    return result;  
}
```

Project 05 — 任务总览与测试

需要实现的 TODO

TODO	内容	难度
1	<code>eval(LObject, Environment)</code>	核心
2	<code>handleLet</code> <code>let/let*</code>	核心
3	<code>handleIf</code> 条件求值	中等
4	<code>handleBegin</code> 顺序求值	简单

明测试一览 (16 个)

分类	测试数
let 基础	3
let* 基础	2
嵌套/遮蔽	3
扩展1: if	5
扩展2: begin	3

截止 下次课开始前

运行: `mvn test`

PART 3 · AI 编程进阶

AI 代码质量管理 · 五轮对话开发框架

AI 辅助 Debug — 四步系统化流程

步骤	提示词重点	说明
1. 提供上下文	粘贴完整 stack trace + 代码	上下文越全，诊断越准
2. 分析根因	"先给出分析，不要修改代码"	先理解问题
3. 最小修复	"只修改 X 方法，不改其他"	控制改动范围
4. 验证修复	"生成正常+异常路径测试用例"	确保修复有效

直接让 AI "修复 Bug" 往往改动范围太大——分步走，你才能理解根因。

AI 生成测试用例

提示词模板

"为以下方法生成 JUnit 4 测试，每个方法至少包含：
1. 正常路径 2. 空结果/边界条件 3. 异常路径
[粘贴方法代码]"

AI 生成示例

```
@Test public void testEvalAtom() { ... }  
@Test public void testEvalEmptyList() { ... }  
@Test(expected = RuntimeException.class)  
public void testEvalUnknownSymbol() { ... }
```

关键提示词："每个方法至少包含正常路径、空结果、异常路径三个测试用例"

AI 辅助代码审查

四点检查提示词

"以高级工程师视角审查以下代码，重点检查：

- 1. 安全性：是否有空指针风险？
- 2. 性能：是否有不必要的循环？
- 3. 可维护性：命名是否清晰？
- 4. 边界条件：空值、越界是否处理？

[粘贴代码]"

AI 审查的局限

AI 擅长	AI 不擅长
语法/拼写错误	业务逻辑正确性
常见安全漏洞	需求合理性
命名规范	架构层面判断

AI 是审查的补充，不是替代——业务逻辑正确性必须由人判断。

五轮对话开发框架

轮次	目标	关键提示词	输出物
1 Plan	架构设计	"规划架构，给出文件结构"	架构图
2 Review	确认无漏	"有没有遗漏？"	修订计划
3 Execute	生成代码	"从 X 文件开始实现"	可运行代码
4 Test	测试验证	"生成 3 种路径测试"	测试通过
5 Document	文档化	"添加 docstring + README"	完整交付

一轮生成的问题：可能有 Bug · 漏需求 · 无测试 · 无文档。五轮框架系统化解决这些。

五轮框架实战 — Plan + Review

轮1 Plan

"我要实现 Project 05 Eval 环境与绑定。
需要支持 let/let*/if/begin。
请规划实现顺序和关键设计点。"

轮2 Review

"请检查以上计划：
– let 和 let* 的环境查找区别是否考虑？
– if 的假值判断是否完整？
– 是否有遗漏的边界情况？"

轮2 审查后补充遗漏——这一步经常能发现 30% 以上的设计盲点。

五轮框架实战 — Execute + Test + Document

轮3 Execute

"从 `handleLet` 开始实现。用 `inplace` 参数统一 `let` 和 `let*`。"

轮4 Test

"为 `handleLet` 生成 JUnit 4 测试：

1. `let` 基础 2. `let*` 前后引用 3. 变量遮蔽 4. 与 `GLOBALS` 结合"

轮5 Document

"为 `Evaluator.java` 所有方法添加 Javadoc, 重点说明 `inplace` 参数。"

每轮控制在 100 行代码以内——超出则主动要求拆分。

当堂练习

当堂练习 · 独立完成 · 25 分钟

练习 1 — HashMap 词频统计 (15 min)

```
import java.util.*;

public class WordFreq {
    // 统计词频
    static HashMap<String, Integer> count(
        String[] words) {
        // 你的代码
    }
    // 找出出现次数最多的词
    static String mostFrequent(
        String[] words) {
        // 你的代码
    }
}
```

提示

- `count` : 用 `getOrDefault` 模式

```
freq.put(w, freq.getOrDefault(w, 0) + 1);
```

- `mostFrequent` : 先调用 `count` , 再遍历 `entrySet()` 找最大值

进阶 (选做)

找出出现次数 ≥ 2 的所有词, 返回 `ArrayList<String>` 。

期望输出

```
count({"a","b","a","c","b","a"})
→ {a=3, b=2, c=1}
mostFrequent(...) → "a"
```

练习 2 — HashSet 集合运算 (10 min)

```
import java.util.*;

public class SetOps {
    // 交集
    static HashSet<Integer> intersect(
        int[] a, int[] b) {
        // 你的代码
    }
    // 并集
    static HashSet<Integer> union(
        int[] a, int[] b) {
        // 你的代码
    }
}
```

期望输出

```
a={1,2,3,4,5}, b={3,4,5,6,7}
intersect → [3, 4, 5]
union      → [1, 2, 3, 4, 5, 6, 7]
```

提示

- 交集：先把 `a` 放入 HashSet，遍历 `b`，`contains` 为 true 加入结果
- 并集：把 `a` 和 `b` 都 `add` 到同一个 HashSet（自动去重）

方法速查

方法	功能
<code>add(e)</code>	添加
<code>contains(e)</code>	是否包含
<code>addAll(c)</code>	批量添加
<code>retainAll(c)</code>	只保留交集

课后作业

必做

1. 完成 Project 05 Env

提交到 SEECODER-Coder，截止下次课前。
四个 TODO: eval · handleLet · handleIf ·
handleBegin

2. AI 辅助实验报告（一段话）

使用五轮框架完成至少一个 TODO，
记录每轮的提示词和 AI 的输出质量

选做

- 用 AI 生成 `handleLet` 的 5 个边界测试
- 思考：为什么 `if` 不先求值所有子表达式？
- 用 AI 生成 Environment 的 PlantUML 类图

预习

下周内容：继承与多态（OOP 核心）

带着问题来：LFunction 和 LFunctionTco
如何共存？

第六次编程实践课

软件工程与计算 I · 2026 · 南京大学

感谢聆听 · 有问题随时提问